

TEE-Crafter: Deploying Unmodified Container Workloads into Hardware Trusted Execution Environments, with Evidence

Vedant Pungliya

2026-08-24

Abstract

TEE-Crafter takes a build context containing a `Dockerfile` and deploys the resulting image into a hardware Trusted Execution Environment on AWS, Azure or Google Cloud, across ten platform backends spanning AMD SEV-SNP, Intel TDX, Intel SGX, AWS Nitro Enclaves, AWS NitroTPM and NVIDIA confidential GPUs. The workload is not modified: attestation, key release, audit logging and evidence collection are the platform’s responsibility rather than the application’s.

This report describes the system, states its trust model per backend — which differs more than the uniform marketing language around “confidential computing” suggests — and reports what has and has not been established on real hardware. It also records nine measured properties of these platforms that contradict their documented or commonly assumed behaviour (Table 7): SEV-SNP launch-measurement semantics on Azure and Google Cloud, three properties of AWS NitroTPM attestation, two classes of supply-chain pin failure, and one access-control gap in policy composition.

Contents

1	Problem	3
2	System model	4
2.1	Inputs and run modes	4
2.2	Pipeline	4
3	Technology stack	5
3.1	Why the CLI runs in a container, and the trap that created	6
4	Platform catalogue	6
5	Attestation evidence formats	6
5.1	AMD SEV-SNP attestation report	7
5.2	Intel TDX and SGX quotes	7
5.3	AWS COSE.Sign1 documents	7
5.4	NVIDIA NRAS Entity Attestation Token	8
6	Trust model, per backend	8
6.1	AWS NitroTPM: measured boot, verified locally	8
6.2	Binding the channel where <code>REPORT_DATA</code> is not ours	9
7	What binds the workload	9
8	The attested handshake	10

9	What the verifier actually checks	10
10	Key release	13
10.1	A conditional Allow gates nothing	13
10.2	A control wired into one call site out of several	14
11	Per-platform notes	15
11.1	nitro-aws	15
11.2	snp-aws	15
11.3	snp-azure	15
11.4	snp-gcp	15
11.5	sgx-azure	15
11.6	tdx-azure	16
11.7	tdx-gcp	16
11.8	gpu-cc-azure	16
11.9	gpu-cc-gcp	16
11.10	gpu-cc-aws	16
11.11	The DEK lifecycle	16
11.12	Sealed application secrets	16
11.13	Compute selection	17
11.14	Resuming a partial deploy	17
12	Empirical findings	18
12.1	On Azure, instance size does not determine host CPU generation	18
12.2	The Azure version suffix is not a second axis	18
12.3	On Google Cloud, a pinned digest can go stale on its own	19
12.4	The NitroTPM PCR bank is SHA-384	19
12.5	Measured boot survives reboot; the AMI plumbing is the fragile part	19
12.6	Measured boot and the launch measurement scale differently	19
12.7	Supply chain: two pinned values that had been wrong for months	20
13	Making the bake fail closed	20
14	Measurement capture and the registry	20
15	Runtime hardening	22
15.1	Three nested boundaries	22
15.2	The systemd layer	22
15.3	Container and handler layers	23
16	The in-TEE audit log	23
16.1	Export, and what the collector receives	24
17	What “unmodified workload” costs	24
18	Network and egress	24
19	Batch mode mechanics	25
20	Teardown and cost hygiene	26
21	Continuous attestation, and failing closed	26
22	Egress posture for GPU attestation	26

23 Evidence produced	27
24 The audit-evidence ledger	27
25 Compliance mapping	28
25.1 The responsibility model is the load-bearing part	28
25.2 Data residency	29
26 Build provenance and supply chain	29
27 Threat model	30
28 Testing strategy, and the ways tests lie	30
29 A worked evidence bundle	31
30 Verification status	32
31 Limitations	33
32 Reproducibility	33
33 The deploy, phase by phase	33
33.1 Phase 1 — build	33
33.2 Phase 2 — package and measure	34
33.3 Phase 3 — generate Terraform	34
33.4 Phase 4 — apply	34
33.5 Phase 5 — verify and collect	34
34 Continuous attestation internals	34
35 Operational runbook: reading a failure	34
36 Cost model for hardware verification	35
37 Positioning	35
38 Future work	36
A Platform capability matrix	37
B Configuration surface	37
C Glossary	37

1 Problem

A confidential-computing deployment is only as useful as the evidence it produces. Running a workload inside an encrypted-memory VM is straightforward; being able to demonstrate afterwards *which* code ran, on *what* hardware, under *whose* signature, is where the work is. In practice teams either modify their application to speak an attestation protocol, or they deploy into a confidential VM and rely on the provider’s assurance without producing anything a reviewer can check.

TEE-Crafter targets the gap between those two. The input is an ordinary container build context. The output is a running workload plus an evidence bundle: an attestation the operator’s client verifies independently, a hash-chained and signed build-provenance trail, the verifier’s own reasoning retained verbatim, and — for one-shot jobs — a signed archive of everything the container wrote.

2 System model

2.1 Inputs and run modes

One input: a directory containing a `Dockerfile`, or a prebuilt OCI image. One choice of run mode:

- **Persistent.** A long-lived service behind a platform-owned attested ingress proxy that terminates RA-TLS. The user container listens on loopback and is unaware of attestation.
- **Batch.** The container runs to completion; every file it wrote is captured by differencing the container filesystem and packaged into a signed archive.

Nine of the ten backends support both modes. Intel SGX under Gramine is batch-only, and the persistent combination is rejected at argument-parse time rather than failing later.

Figure 1 shows what differs between them. The important structural point is that in both modes the user container sits behind a boundary it does not participate in: it binds loopback, and the attested surface belongs to the platform.

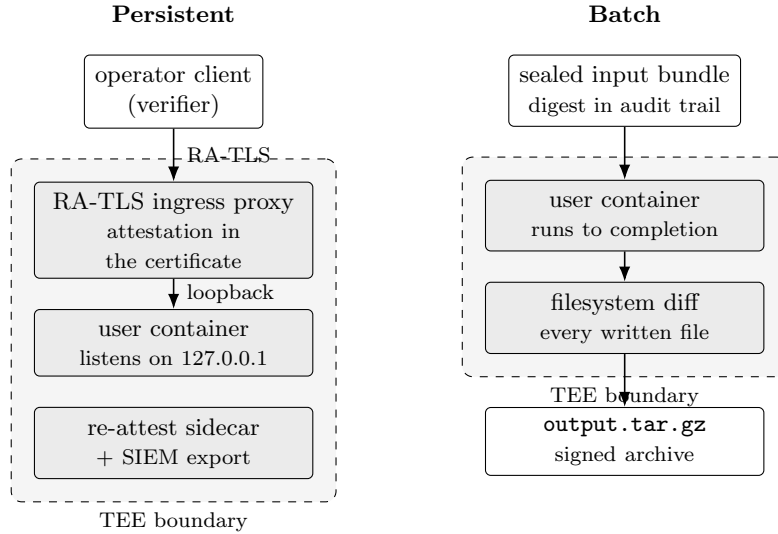


Figure 1: The two run modes. In both, the user container is unmodified and unaware of attestation. Persistent mode attests a live channel; batch mode attests a completed computation and signs its output.

2.2 Pipeline

Every deployment runs the same five phases, shown in Figure 2. Machine images are baked in a separate, deliberately infrequent step, so a deploy never installs a toolchain on a production instance.

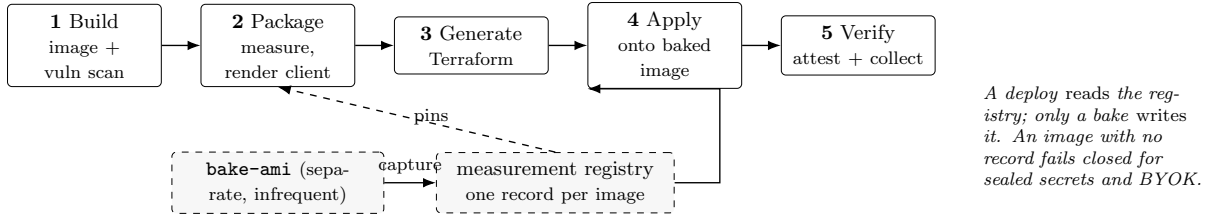


Figure 2: The deploy pipeline, and the bake that feeds it. Phase 2 renders the verifier client with the pins the bake captured, so the client that checks the deployment is built from measured values rather than hand-entered ones.

3 Technology stack

Nothing here is novel technology; the engineering is in the composition and in what is pinned. Every version below is fixed in the CLI’s own container image or its requirements file, so a deploy is not at the mercy of whatever the package archives served that morning.

Component	Pinned at	Why it is in the trusted path
Python	3.12	Runs the CLI, the verifier clients and the in-TEE runtime modules. 3.10 is also present for the Gramine toolchain.
Terraform	1.15.9	Sole means of provisioning. Ten platform templates, validated in CI.
Docker / BuildKit	host	Builds the workload image and computes the digest that gets bound into attestation.
Trivy	apt (signed repo)	Vulnerability gate. Production default blocks fixable CRITICAL/HIGH.
cryptography	50.0.0	All certificate-chain and signature verification on the client side.
cbor2	5.9.0	COSE_Sign1 decoding for Nitro and NitroTPM documents.
PyJWT[crypto]	2.13.0	NVIDIA NRAS EAT and Microsoft MAA token verification.
boto3	1.42.54	AWS control plane, KMS, SSM.
google-cloud-kms	3.16.0	GCP key operations.
click	8.1.8	CLI surface, including the <code>envvar=</code> bindings that make every flag settable from <code>.env</code> .
Gramine / GSC	v1.9, commit 0b2ba931	SGX enclave packaging. Pinned by commit, not tag.
snpguest	v0.7.0, commit 49494ff7	SEV-SNP report retrieval and endorsement-certificate fetch. §12.7 is about this pin.
nitro-tpm-attest	commit 441fe310	NitroTPM attestation documents. Built static via upstream’s Alpine builder.
AWS Session Manager	apt (Amazon)	Keyless, agentless remote exec — no inbound SSH on AWS.

Table 1: The pinned toolchain. Two of these pins were wrong for months (§12.7); the lesson is in §13.

Three deliberate absences. There is no agent installed on the workload: the attested proxy, the re-attestation sidecar and the audit logger are platform code staged at deploy time, not something the user’s image must import. There is no orchestrator — one deployment is one VM or one enclave, because a scheduler that can move a workload between hosts undermines the thing being attested. And there is no long-lived control plane: the CLI runs, produces

evidence, and exits.

3.1 Why the CLI runs in a container, and the trap that created

The CLI re-executes itself inside its own image so that Terraform, the cloud CLIs, Trivy and Gramine are the pinned versions rather than whatever the operator has. The source is baked into that image rather than bind-mounted.

That is a correctness hazard and it fired. The image-reuse path returned early whenever the tag existed, so editing code and re-running silently exercised the *previous* build, which produced two clean-looking **sgx-azure** runs executing deleted code, and the error they printed pointed at the wrong cause entirely. The fix is a fingerprint over every file that gets baked in — not only the Python — compared against a label on the image, with a rebuild when they differ. It is recorded here because the failure mode is invisible by construction: a stale image looks exactly like a fresh regression.

4 Platform catalogue

Ten backends, four TEE technologies, three clouds. Table 2 is the operational view; §5 gives the evidence formats.

Platform	Cloud	TEE	Identity	Modes	Remote exec
nitro-aws	AWS	Nitro claves	En-PCR0	both	SSM
snp-aws	AWS	AMD SEV-SNP	MEASUREMENT	both	SSM
snp-azure	Azure	AMD SEV-SNP	MEASUREMENT	both	Bastion SSH
snp-gcp	GCP	AMD SEV-SNP	MEASUREMENT	both	IAP SSH
sgx-azure	Azure	Intel SGX	MRENCLAVE	batch only	Bastion SSH
tdx-azure	Azure	Intel TDX	MRTD (via MAA)	both	Bastion SSH
tdx-gcp	GCP	Intel TDX	MRTD	both	IAP SSH
gpu-cc-azure	Azure	SEV-SNP + H100	MEASUREMENT + EAT	both	Bastion SSH
gpu-cc-gcp	GCP	TDX + H100	MRTD + EAT	both	IAP SSH
gpu-cc-aws	AWS	NitroTPM + H100	PCR4/7 + EAT	both	SSM

Table 2: The ten backends. “Identity” is what a pin actually names — note that it is a different *kind* of value in each family, which is why one cannot be substituted for another.

Two asymmetries in that table matter more than they look. Only AWS offers keyless remote execution (SSM), so the AWS platforms open no inbound SSH at all, while Azure and GCP require a bastion or an IAP tunnel and therefore a key with a lifetime. And **sgx-azure** is batch-only: an SGX enclave under Gramine has no practical story for a long-lived attested listener here, so the persistent combination is refused at argument-parse time rather than failing after an instance is billing.

5 Attestation evidence formats

A verifier parsing the wrong offset produces a structurally valid wrong answer. Each layout below is parsed identically by the in-TEE producer and the operator’s client, from shared constants.

5.1 AMD SEV-SNP attestation report

A fixed 1184-byte structure. The fields this system reads:

Offset	Size	Field and how it is used
0x00	4	VERSION.
0x08	8	POLICY. Bit 16 is SMT-allowed; debug and migration bits are checked and refused.
0x50	32	REPORT_DATA's sibling fields begin here; the guest-controlled block carries the binding digest.
0x90	48	MEASUREMENT — the SHA-384 launch digest. This is the value the registry pins.
0x1A8	8	PLATFORM_INFO. Bit 5 is ALIAS_CHECK_COMPLETE and is fatal if unset — it confirms the AMD Secure Processor completed the memory-aliasing mitigation for AMD-SB-3015 (CVE-2024-21944).
—	8	REPORTED_TCB / COMMITTED_TCB. Firmware SVN in bits 55:48 must meet a per-generation floor, and COMMITTED $\leq$ REPORTED is enforced as an anti-rollback check.

Table 3: SEV-SNP report fields consumed. The AMD-SB-3015 and anti-rollback checks are fatal, not advisory: a platform that has not completed alias checking is exactly the platform where memory encryption can be subverted.

The signature is ECDSA P-384 with SHA-384 (`sig_algo == 1`, enforced rather than inferred), verified against the VCEK or VLEK, which chains ASK/ASVK $\rightarrow$ ARK.

5.2 Intel TDX and SGX quotes

MRTD is a 48-byte SHA-384 in the TDREPORT's TDINFO structure. The framing differs by source: **MRTD sits at offset 184 in a DCAP quote and 560 in an Azure vTPM HCL report.** Reading the wrong offset yields 48 bytes of an adjacent field — structurally valid (96 hex characters, storable, enforceable) and guaranteed to fail every verification. Both readers are implemented, tried in order, and the framing used is recorded.

Beyond the quote signature, the DCAP path verifies the PCK certificate chain to Intel's pinned SGX Root CA, checks the Quoting Enclave report's binding to that chain's leaf, and evaluates TCB status against collateral fetched from Intel's Provisioning Certification Service. The ordering is load-bearing: the QE report signature must consume the leaf that chain verification validated, not a re-parsed one, and TCB evaluation must follow signature verification. Those edges are pinned by test because they are invisible in review.

5.3 AWS COSE_Sign1 documents

Both Nitro Enclaves and NitroTPM produce a CBOR COSE_Sign1: a four-element array of protected header, unprotected header, payload and signature. The signature covers a canonical `Sig_structure` of `["Signature1", protected, external_aad, payload]`, and the algorithm is read from the protected header rather than assumed. Real NitroTPM documents carry `{1: -35}`, ES384, with a 96-byte raw `r||s` signature that must be re-encoded as DER before verification.

The payload is a map. For NitroTPM the keys observed on hardware are `cabundle`, `certificate`, `digest` (the string "SHA384"), `module_id`, `nitrotpm_pcrs` (integer keys, 48-byte values), `nonce`, `public_key`, `timestamp` in milliseconds, and `user_data`.

5.4 NVIDIA NRAS Entity Attestation Token

A JWT — in practice a nested one: an outer token whose claims contain per-GPU sub-tokens. Verification pins the NRAS signing intermediate byte-for-byte as `x5c[1]`, refuses algorithm confusion, and checks the `eat_nonce` against a nonce the client recomputes locally from the server’s published ECDH key and salt. That last step is what makes the token about *this* session rather than any session: NVIDIA signs the nonce, so a replayed token fails.

The claims consumed include GPU model, driver and VBIOS versions, secure-boot state, debug status (which must be disabled) and whether the attestation key is AMD-rooted.

6 Trust model, per backend

The single most important thing this report can convey is that “confidential computing” is not one guarantee. Table 4 states who actually signs the evidence on each backend.

Backend	Evidence	Root of trust, and its limit
<code>nitro-aws</code>	COSE_Sign1 document	AWS Nitro Enclaves root, pinned in-repo. Key release bound to PCR0/1/2 by KMS condition keys.
<code>snp-aws</code>	SEV-SNP report	AMD, via VCEK. Key release additionally bound to NitroTPM measured boot (§6.1).
<code>snp-azure</code>	SEV-SNP report + vTPM quote	AMD, via VCEK — but <code>REPORT_DATA</code> is fixed by the Hyper-V paravisor, so the channel key is bound through the paravisor’s own attestation key (§6.2).
<code>snp-gcp</code>	SEV-SNP report	AMD, via VCEK.
<code>sgx-azure</code>	DCAP quote	Intel, via the Provisioning Certification Service. Identity is <code>MRENCLAVE</code> .
<code>tdx-gcp</code>	TDX DCAP quote	Intel.
<code>tdx-azure</code>	MAA token	Microsoft, not Intel. The guest cannot produce a DCAP quote — no Quoting Enclave is reachable under the paravisor — so the host brokers one and Microsoft Azure Attestation’s verdict <i>is</i> the attestation.
<code>gpu-cc-azure</code>	SNP report + NVIDIA EAT	AMD and NVIDIA.
<code>gpu-cc-gcp</code>	TDX quote + NVIDIA EAT	Intel and NVIDIA.
<code>gpu-cc-aws</code>	NitroTPM document + NVIDIA EAT	AWS and NVIDIA. No CPU-TEE: the boot chain is attested, host memory is not encrypted (§6.1).

Table 4: Root of trust per backend. `tdx-azure` roots in Microsoft rather than Intel. `gpu-cc-aws` roots in AWS for measured boot only — see §6.1.

Figure 3 draws the four distinct chain shapes behind that table. They are worth seeing side by side because the number of parties in each differs, and so does who can break it.

6.1 AWS NitroTPM: measured boot, verified locally

A NitroTPM attestation document is a COSE_Sign1 signed by the Nitro Hypervisor. Its `cabundle` roots at `CN=aws.nitro-enclaves` — byte-for-byte the same certificate used for Nitro Enclaves attestation, SHA-256 641a0321a3e244efe456463195d606317ed7cdcc3c1756e09893f3c68f79bb5b. This contradicts the common assumption that NitroTPM uses a separate key hierarchy requiring a root not published for guest use.

Measured against a 5163-byte document from a live instance: the chain is five certificates deep (TPM leaf → instance → zonal → region → root), every link’s signature validates, and

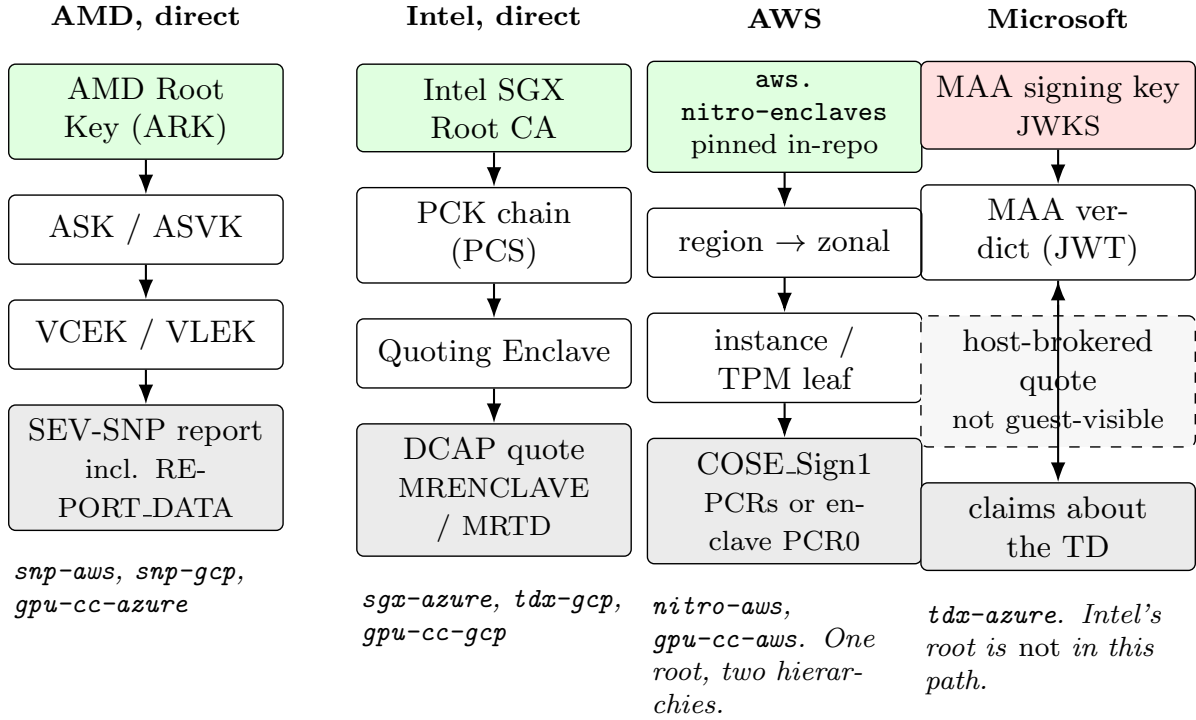


Figure 3: The four chain shapes. Green marks the anchor a verifier pins. In the Microsoft case the pinned anchor is a service’s signing key rather than a silicon vendor’s root, so the verdict is a statement by Microsoft about Intel hardware rather than a signature from Intel hardware.

the COSE_Sign1 signature verifies under the TPM leaf’s P-384 key using the algorithm from the protected header ($\{1: -35\}$, ES384). The consequence is that the document is verifiable locally, against a pinned root, with no AWS credentials and no delegation to KMS.

The same document is independently usable as a KMS **Recipient** to gate key release (§10).

Scope: this yields measured boot, not memory encryption. AWS GPU instances have no CPU-TEE — host RAM is readable by the hypervisor and the CPU-GPU PCIe link is not TEE-encrypted. Measured boot establishes what booted, not that its memory stays private.

6.2 Binding the channel where REPORT_DATA is not ours

On Azure SEV-SNP the paravisor fixes REPORT_DATA to $\text{sha256}(\text{runtime_data})$, so the guest cannot place its own key hash under AMD’s signature. A TPM quote alone does not substitute: an adversary replaying a captured SNP report can mint an attestation key and sign a quote committing to their own key, and the result is self-consistent.

What closes it is the runtime data itself. AMD signs REPORT_DATA; REPORT_DATA is the digest of the runtime-data document; and that document names the paravisor’s attestation key. So the chain runs as in Figure 5, and the server signs its quote with that key rather than one it generated. The client fails closed unless this holds. Verified on hardware for *snp-azure* and *gpu-cc-azure*.

7 What binds the workload

The launch measurement identifies the *machine*, not the application. On a confidential VM it covers initial guest memory — firmware, boot configuration, one vCPU state per virtual processor — and not the contents of the OS disk, which is attached after launch.

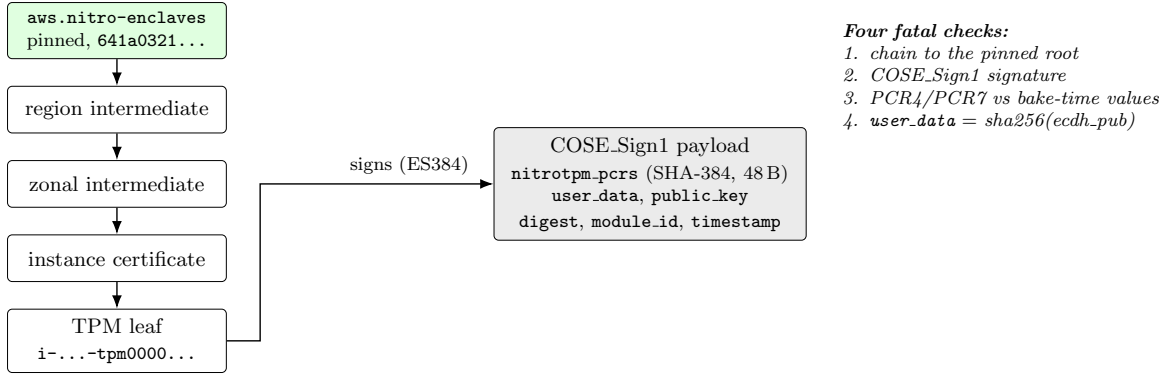


Figure 4: A NitroTPM attestation document and how the client verifies it. Check 4 is the channel binding: without it a genuine document from any other instance in the account would pass.

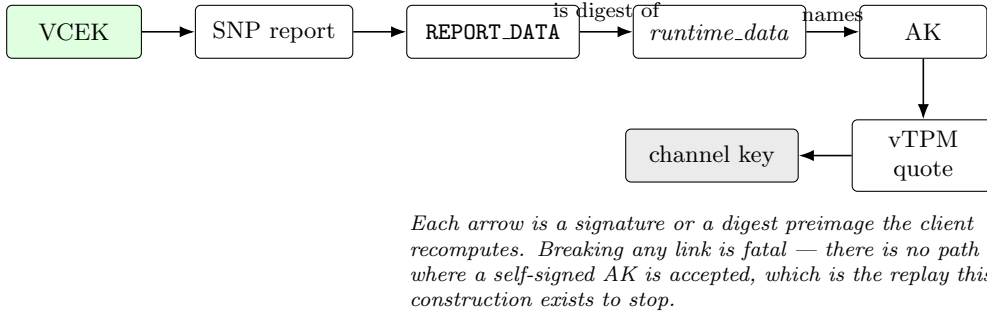


Figure 5: The Azure paravisor binding chain. AMD’s signature reaches the channel key transitively, through a document that names the attestation key.

The workload is bound separately, by hashing the container image digest into the attestation alongside the server’s ephemeral public key and a commitment to the runtime audit log’s chain key. The verifier recomputes that preimage and compares. Figure 6 shows why the encoding is length-prefixed rather than concatenated.

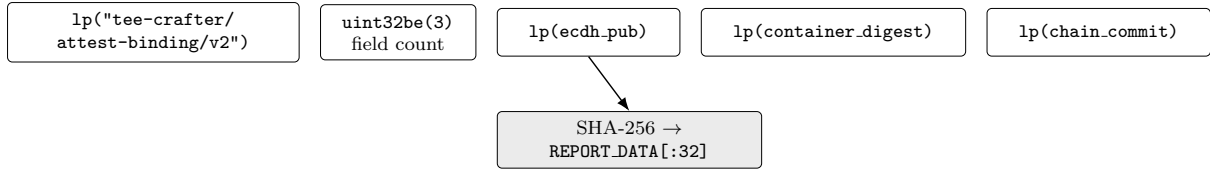
Read the launch measurement as “a genuine confidential VM on the firmware I expect” and the container digest as “running the code I expect”; neither substitutes for the other.

8 The attested handshake

Figure 7 is the persistent-mode exchange. Every check on the client side aborts rather than warns, and the ordering matters: several checks are worthless unless their predecessor has already run, so the order is asserted by test rather than left to reading.

9 What the verifier actually checks

“Attestation verified” is a claim with wide variance behind it, so this is the inventory. Every row is fatal — there is no warn-and-continue path in a client — and the ordering constraints noted are asserted by test because they are invisible in review.



lp(x) is uint32be(len(x)) || x. Raw concatenation would be ambiguous: ("ab", "cd") and ("abc", "d") hash identically, so evidence minted for one field split could be presented as satisfying another. The version label and the field count are hashed in too, so a v1 preimage cannot be reinterpreted as v2 and a short field list cannot be padded into a longer one.

Figure 6: The attestation binding preimage. The verifier recomputes this byte-for-byte, which is why the version label is hashed data rather than a comment.

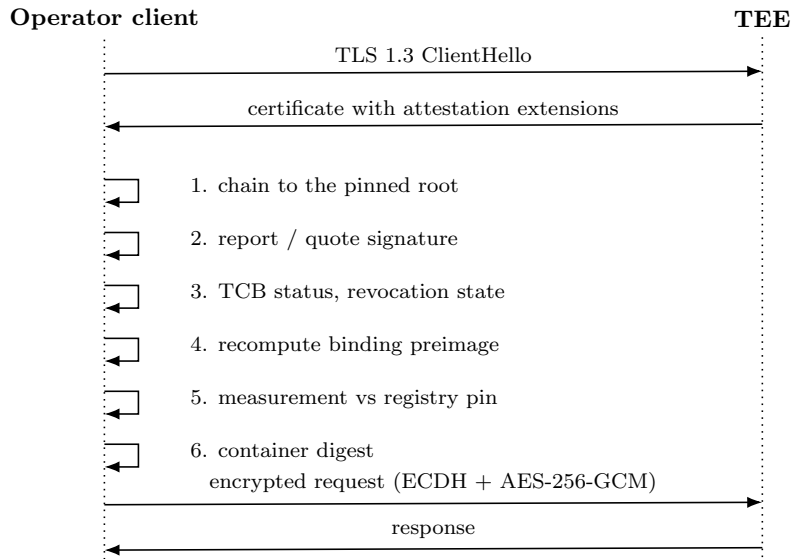


Figure 7: Persistent-mode RA-TLS. The attestation travels in X.509 extensions on the server certificate, so no application protocol change is required; the client’s five self-checks are all fatal.

Check	Families	Why it is not redundant
Certificate chain to a pinned root	all	Without a pinned anchor a chain proves only internal consistency.
Chain validity windows	all	An expired leaf on a valid chain is still expired.
Signature at every link	all	Name chaining without signature checking accepts any certificate claiming the right issuer.
Top-of-chain equals the pinned root	all	Byte comparison, not subject-name comparison — anyone can put CN=ARK-Milan on a self-signed cert.
Key, not name, pinned for AMD	SNP	The ARK is pinned by SubjectPublicKey-Info digest for exactly that reason.
Report/quote signature algorithm	SNP, TDX	Enforced as a value (<code>sig_algo == 1</code>), not inferred from the key — algorithm confusion is otherwise available.

... continued

Check	Families	Why it is not redundant
COSE algorithm from the protected header	AWS	Read, then checked against an allowlist; an absent or unknown value is refused rather than defaulted.
Report signature over the report	SNP, TDX	—
Guest policy: debug disabled	SNP	A debug-enabled guest is inspectable by the host, which voids the property being at-tested.
Guest policy: migration disabled	SNP	—
ALIAS_CHECK_COMPLETE set	SNP	AMD-SB-3015 (CVE-2024-21944). Without it, memory-aliasing mitigation did not complete.
Firmware SVN floor	SNP	Per-generation minimum; blocks known-vulnerable firmware.
COMMITTED_TCB $\leq$ REPORTED_TCB	SNP	Anti-rollback.
PCK chain to Intel's root	TDX, SGX	—
QE report bound to the PCK leaf	TDX, SGX	Must consume the leaf that chain verification validated , not a re-parsed one. Ordering asserted.
TCB status evaluation	TDX, SGX	Must follow signature verification; evaluating the TCB of an unverified quote proves nothing.
Launch measurement vs registry pin	CVM	The pin comes from a booted capture; hand-entered pins are rejected.
Measured-boot PCRs vs bake values	*-aws	A different property from the launch measurement, and not a substitute.
Binding preimage re-computed	all	Length-prefixed, version-labelled; proves the evidence is about <i>this</i> channel.
Container digest	all	The only check that speaks to the application rather than the machine.
Audit chain-key commitment	all	Folded into the hardware-signed binding, so a replaced log cannot be self-consistent.
TLS SPKI matches published value	all	Catches cert-vs-key inconsistency and MITM fabrication.
NRAS token signature + x5c[1] pin	GPU	Intermediate pinned byte-for-byte.
NRAS eat_nonce re-computed locally	GPU	Must use the locally recomputed nonce , not the one the server offered — otherwise a replayed token passes.
GPU debug status disabled	GPU	—
GPU secure-boot state	GPU	—
NitroTPM user_data = sha256(ecdh_pub)	gpu-cc-aws	Without it, a genuine document from any other instance in the account passes.

MAA token issuer al-lowlist	tdx-azure	The verdict is the attestation, so the issuer is the root.
-----------------------------	-----------	--

Two things this table is meant to make visible. First, the count varies a lot by family: an Intel DCAP path runs roughly twice as many independent checks as an AWS COSE path, not because it is more secure but because the evidence is assembled from more parties. Second, four rows carry explicit *ordering* constraints, and those are the ones where a correct check can be worthless — which is why they are pinned by source-order assertions rather than trusted to review.

10 Key release

Customer-managed key release starts, on every provider and platform, at *identity*-gated: the custodian checks who is asking, not what they are running. Three of the combinations can be upgraded to custodian-enforced attestation by supplying the right policy conditions. Table 6 states which, and what each upgrade actually binds.

Provider	Platform	Upgrade path, and what it binds
aws-kms	nitro-aws	<code>kms:RecipientAttestation:PCR{0,1,2}</code> . Binds the enclave image. The strongest of the three. Verified on hardware.
aws-kms	snp-aws, gpu-cc-aws	<code>kms:RecipientAttestation:NitroTPMPCR{4,7}</code> . Binds <i>measured boot</i> , not the SEV-SNP launch measurement — AWS publishes no condition key for the latter. Verified on hardware.
azure-kv / azure-skr	snp-azure, tdx-azure, gpu-cc-azure	Secure Key Release with <code>x-ms-sevsnpvm-launchmeasurement</code> bound. The shipped default pins only <code>x-ms-attestation-type</code> , which any SEV-SNP CVM in any tenant satisfies. Verified on hardware for snp-azure .
gcp-kms	snp-gcp, tdx-gcp, gpu-cc-gcp	A Confidential Space workload-identity attribute condition on the IAM binding. Cloud KMS AAD is <i>not</i> an authorisation input — it is a plain AEAD binding.
external-hsm	any	Delegated. TEE-Crafter forwards the evidence and cannot assert what the gateway does with it.

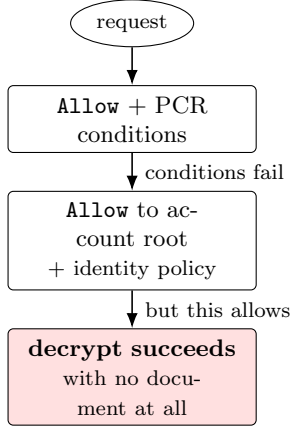
Table 6: Key-release gating. The distinction that matters to an auditor is whether the *custodian* refuses, or whether the deploying code checks a measurement in-process on the host it is defending against. The latter is advisory and is labelled as such in the audit ledger.

10.1 A conditional Allow gates nothing

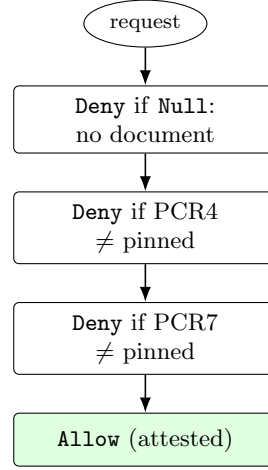
AWS KMS authorises a request if *any* statement allows it. A key policy whose **Allow** carries `kms:RecipientAttestation:NitroTPMPCR4` conditions is therefore bypassed by the combination of account-root delegation and an identity-based `kms:Decrypt` on the caller — both of which are default configurations. Measured: with such a policy in place, `kms:Decrypt` presenting *no* attestation document succeeds, as does one presenting a mismatched PCR4. The policy is indistinguishable from a correctly gated one by inspection of its **Allow** statements alone.

Explicit **Deny** is what closes it, and because condition keys inside a single condition block are AND-ed, each pinned PCR needs its own **Deny** statement. Figure 8 shows the evaluation before and after.

Before: conditional Allow only



After: explicit Deny



One Deny per PCR, because conditions within one block are AND-ed — a single block would only deny a request that mismatched every pinned register at once.

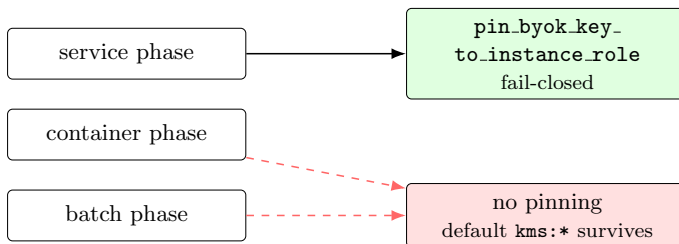
Figure 8: KMS policy evaluation. A **Deny** is unconditional in effect: it overrides every **Allow**, including one inherited through account-root delegation. Post-fix behaviour, measured: correct PCRs succeed, absent document denied, wrong PCR4 denied.

Two operational details fall out of the same runs. NitroTPM attestation documents **expire after five minutes** — KMS answers `ValidationException: exceeded the five-minute age limit` — so any flow that generates a document and then does slow work before calling KMS fails. And the response is CMS EnvelopedData (RFC 5652), not a raw RSA-OAEP blob: a consumer that reads `KeyEncryptionAlgorithm=RSAES_OAEP_SHA_256` and performs one RSA operation fails on the symmetric layer.

10.2 A control wired into one call site out of several

Pinning the KMS key policy to the role Terraform creates is fail-closed code: it refuses the deploy if the policy cannot be narrowed, since an unpinned key is readable by any matching role in the account. It was reachable from one call site — the `snp-aws` service phase.

The container flows (`--batch` and the persistent container path) did not call it. Effect: a deploy passing `--byok aws-kms` with `unwrap=aws_nitrotpm_recipient` completes successfully against a key retaining its default policy — no measurement condition, no principal narrowing, no diagnostic. Confirmed by reading the applied policy after such a run: a single `Allow kms:*` statement.



The control was correct, tested and fail-closed — in the one flow that reached it. Coverage of the function said nothing about coverage of the paths.

Figure 9: The gap. Both dashed edges now route through the same fail-closed wrapper.

This is the same class of defect as §10.1 and as the inert `gpu-cc-gcp` check in §28: the control exists, is individually correct, and is not on the path the request actually takes. Unit tests cannot see it, because each unit passes. What finds it is either a hardware run or an explicit assertion that every flow reaches the control — and the cheap version of that is to have one wrapper rather than one call site.

11 Per-platform notes

The families share implementation; the per-platform differences are below.

11.1 nitro-aws

Two AMIs, not one: the parent instance and the enclave image are separate artefacts and both are pinned. PCR0 is computable at build time from the EIF, so this platform needs no post-bake capture. It is the reference implementation for attestation-gated key release — `kms:RecipientAttestation:PCR{0,1,2}` is a real condition key that KMS evaluates, and the enclave’s ephemeral public key is what KMS re-encrypts to. Continuous attestation is only meaningful here because the SIEM exporter runs *inside* the enclave; a host-side sidecar would write its health file where the enclave cannot read it, and the gate would pass everything.

11.2 snp-aws

SSM-only, no inbound SSH. Secure Boot is enrolled at bake time into the AMI’s `UefiData` because AWS exposes no `RunInstances` parameter for it. The launch measurement varies per vCPU tier, so the registry stores one digest per tier; measured-boot PCRs do not, so they are stored once. This is the platform where §6.1, §10.1 and §12.7 all apply.

11.3 snp-azure

The paravisor fixes `REPORT_DATA`, so channel binding runs through the runtime-data document and the paravisor’s attestation key (§6.2). There is no `/dev/sev-guest`: the SNP report is read from the vTPM at NV index `0x01400001` as an HCL blob with an HCLA header wrapping the 1184-byte report. Key release via `azure-skr` is the only Azure choice that yields a usable DEK, because Key Vault wraps the released key to a vTPM-sealed KEK that no Python process holds. That tool’s wire contract is asymmetric — base64 in, raw bytes out — and treating it as symmetric corrupts the key silently.

11.4 snp-gcp

`min_cpu_platform` makes the host generation selectable, which is the one cloud where it is. This is where a pinned digest was observed going stale without the image changing.

11.5 sgx-azure

Batch-only. Identity is `MRENCLAVE`, computed from the Gramine manifest at build time. The enclave is packaged with GSC pinned by commit. Input and output traverse host-visible paths the enclave does not verify, which is the honest limitation of this backend and is warned about on every run rather than suppressed. Kernel lockdown stays enforced because the SGX userspace stack comes from signed Intel and Gramine repositories and needs no unsigned kernel module.

11.6 tdx-azure

The one backend whose root of trust is not a silicon vendor. No Quoting Enclave is reachable under the paravisor, so the host brokers a quote and Microsoft Azure Attestation’s signed verdict *is* the attestation. Everything else here — the binding, the digest, the audit chain — works as elsewhere; the difference is who you are trusting, and no amount of client-side care changes it.

11.7 tdx-gcp

A real Intel DCAP path: quote, PCK chain to Intel’s pinned root, QE report binding, TCB evaluation against live PCS collateral. Note the MRTD framing trap in §5 — offset 184 in a DCAP quote versus 560 in an HCL report.

11.8 gpu-cc-azure

Dual attestation with both halves genuinely rooted: SEV-SNP for the CPU, NVIDIA NRAS for the H100, over an encrypted PCIe link. Secure Boot defaults *off* here for driver-stack reliability under kernel lockdown, not because the hardware forbids it; enabling it requires confirming a signed NVIDIA module exists for the exact kernel release. This is the platform on which strict NRAS egress was proven (§22).

11.9 gpu-cc-gcp

Dual attestation with Intel TDX and NVIDIA. The vTPM PCR bundle it publishes is *unsigned* — a tripwire, with the TDX quote as the real anchor. Two defects would have failed the first hardware run and were fixed before it: the image did not install the TPM tooling its own runtime shells out to, and no reference PCR set was ever captured or passed to the client.

11.10 gpu-cc-aws

No CPU-TEE. Attested via NitroTPM measured boot plus NVIDIA NRAS (§6.1); host memory is not encrypted and the PCIe link is not TEE-protected. The deploy requires an explicit acknowledgement of the weaker model before it will proceed, and the measurement it pins is PCR4/PCR7 rather than a launch digest — the two are not interchangeable, and a caller passing a launch measurement here gets a warning rather than silence.

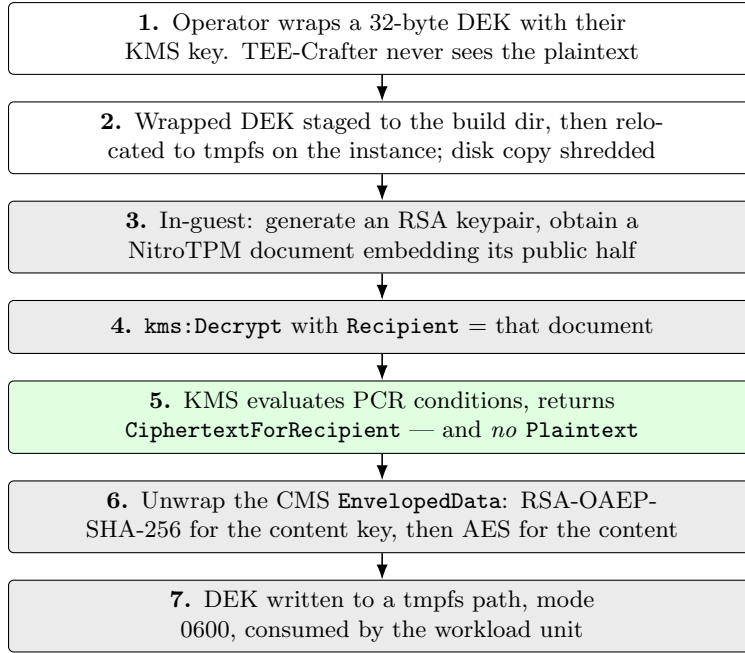
11.11 The DEK lifecycle

Key release is the part most likely to be described loosely, so this is the whole path for the strongest AWS case.

Three failure modes, each producing a structurally valid wrong result. Requesting a **Recipient** without a public key returns **Plaintext** — key material crossing the wire outside the TEE’s control, which is the opposite of the intent, so the code refuses to build that request. Attaching PCR conditions to a key while the runtime still uses **direct.bytes** takes BYOK from weakly-gated to non-functional, since KMS then denies every request lacking a document; the two must ship together, which is why the pinning step reads the registry and refuses when PCRs are absent rather than pinning half of it. And a five-minute-old document fails regardless of correctness.

11.12 Sealed application secrets

Distinct from BYOK and often confused with it. **--secrets-env** takes a **.env** of application configuration. Without BYOK it is baked into the measured image, which is fine for configuration and wrong for secrets — it is in the image, and the image is only as private as the



Step 5 is the point of the exercise: because the response is encrypted to a key generated inside this process, a replayed document cannot be paired with a usable response. Step 6 is where a naive implementation fails — the blob is CMS, not raw OAEP, so a single RSA operation gets you the content key and then nothing.

Figure 10: Attested DEK release on `snp-aws` / `gpu-cc-aws`. The absence of `Plaintext` in the response is what distinguishes an attested call from an identity-gated one, and is worth asserting on.

disk. With `--byok aws-kms` or `gcp-kms` the file is envelope-sealed to the DEK and opened only inside the attested TEE.

The gate: a sealed-secrets deploy against an image with no measurement pin is refused. Releasing secrets to a TEE whose launch state nothing vouched for is precisely the case the pin exists for.

11.13 Compute selection

Instance shape is chosen from a per-platform catalogue rather than accepted freely, because most shapes in a family do not support the TEE. The catalogue enumerates concrete shapes per TEE and cloud with their vCPU and memory, and the deploy refuses a shape outside it rather than failing at `RunInstances`. `--cpu` / `--ram` pick the smallest qualifying shape; `--instance-type` names one exactly; the `TEE_CRAFTER_COMPUTE_OVERRIDE_*` variables exist for automation.

Interaction with measurement pinning: on SEV-SNP a shape change moves the launch digest, so a resize requires a registry entry for the target tier. The deploy checks this before provisioning and reports which tiers were captured.

11.14 Resuming a partial deploy

`deploy-from-build` finishes or re-drives an already-staged build directory, dispatching on the platform recorded in its manifest and restoring the `TF_VAR_*` environment from it — so a resumed apply plans what the original apply planned. A directory recording no platform is refused rather than guessed at.

Before anything runs, the directory’s provenance hash chain and Ed25519 signature are

verified. That check is not ceremony: the build directory is read straight off local disk and its artefacts are what get measured into the TEE, so an unverified resume is an unmeasured input.

12 Empirical findings

Table 7 indexes the results; the subsections give each in full.

Measured property	Method	Consequence
Azure instance size does not determine host CPU generation	bake	Generation is not inferable from a SKU. Probes grouped by inferred label attribute host-generation differences to vCPU tier.
The Azure v5/v6 suffix does not affect the launch digest	bake	A deploy gate keyed on the suffix rejects working configurations.
GCP launch digests track host firmware	re-measure	A correct pin can fail with no change to image or deployment.
NitroTPM documents chain to the Nitro Enclaves root	decode	gpu-cc-aws CPU evidence is locally verifiable; no KMS delegation required.
A conditional KMS Allow is not a gate	negative test	Requires one explicit Deny per pinned PCR.
The NitroTPM PCR bank is SHA-384	decode	A SHA-256 pin denies the legitimate caller.
Measured-boot PCRs are vCPU-independent; launch measurements are not	bake	A PCR-pinned key policy survives instance resize; a measurement-pinned one does not.
Non-fatal build steps mask invalid supply-chain pins	fail-closed gate	A nonexistent commit pin and an intermediate-instead-of-root fingerprint both persisted silently; no snp-aws bake installed an AMD endorsement chain.
A fail-closed control reachable from one of three flows	end-to-end run	--byok on container flows left the key policy unpinned.

Table 7: Measured properties and their consequences. None were detected by unit tests; the suite passed in every case.

Each of these was obtained by measuring rather than reasoning, and each contradicts an assumption that looked safe.

12.1 On Azure, instance size does not determine host CPU generation

The SEV-SNP launch measurement folds in host firmware and microcode, so it varies with the host CPU generation. It is natural to infer the generation from the instance size — Azure’s v5 and v6 suffixes suggest exactly that — and this project did so.

It is wrong. A bake measured **Standard_DC2as_v5** and **Standard_DC4as_v5**, both of which a size-based mapping calls Milan, and both reported AMD EPYC 9V74, which is Genoa. Because the earlier code grouped probes by an inferred generation, a host-generation difference had previously been recorded as a vCPU-tier difference, and successive bakes of one image family disagreed with each other in a way that looked like a tooling bug.

The fix is to read the CPU model from the measuring VM and label the generation as observed or inferred, never silently either.

12.2 The Azure version suffix is not a second axis

The obvious follow-up question — whether v5-on-Genoa and v6-on-Genoa produce the same digest — had no data behind it, and the answer determines whether a deploy gate keyed on the suffix is load-bearing or merely cautious. Once **standardDCasv6Family** quota was granted, a

bake on measurement of `Standard_DC2as_v5`, `Standard_DC4as_v5` and `Standard_DC4as_v6`: all three produced the *same* digest, with `cpu_gen` observed as `genoa` each time.

So the determinant is the host CPU generation — precisely the thing an Azure instance type cannot select — and not the version suffix. A refusal keyed on the suffix was written, measured against, and then removed, because it rejected deploys that work. vCPU count remains an input, since SEV-SNP measures one VMSA per virtual processor.

12.3 On Google Cloud, a pinned digest can go stale on its own

A single image, booted twice on the same machine type with the same minimum CPU platform a day apart, produced two different launch measurements (`0e017d2f...` then `2d24cf96...`); the second value matched two other, independently built images. The measuring VM reported AMD EPYC 7B13 on both occasions, so this is not a generation change but host firmware or microcode beneath one generation.

Two conclusions follow. First, the measurement is genuinely disk-independent there: three different images converge on one digest when the host is consistent. Second, and operationally more important, a correctly pinned image can begin failing attestation *because the provider updated host firmware*, with nothing about the image or the deployment having changed. On that platform a mismatch should prompt re-measurement before suspicion — while remaining a mismatch, never something to paper over by widening an allowlist.

Analysis note: collapsing digests to a set reports “every bake differs” for a population where two of three agree exactly. Partitioning images by digest and isolating the outlier is what makes the hypothesis testable.

12.4 The NitroTPM PCR bank is SHA-384

A NitroTPM attestation document reports `digest: SHA384` and carries 48-byte PCR values. A key policy or client pin written with 32-byte SHA-256 values does not fail loudly — it denies the *legitimate* caller, which is how this surfaced. The reading code now rejects a 32-byte value outright rather than storing something that can only ever mismatch. By contrast the Google Cloud vTPM bundle on `gpu-cc-gcp` genuinely is SHA-256, because that is the bank the server reads; the two must not be compared across banks.

12.5 Measured boot survives reboot; the AMI plumbing is the fragile part

PCR4 and PCR7 were byte-identical across a real reboot, and a second `kms:Decrypt` succeeded against the unchanged key policy. This is the result that makes pinning viable rather than an outage waiting to happen — AWS’s own guidance assumes an `erofs` plus `dm-verity` root, which these images do not use.

The fragility is elsewhere, in how the AMI is registered. `RegisterImage` is the only API that can set `TpmSupport` (`CreateImage` cannot), and it *silently drops* `UefiData` unless that is passed explicitly — which disables Secure Boot on the resulting image and therefore changes PCR7. An AMI can thus be produced that looks correct, boots, and attests to a PCR7 nobody expected.

12.6 Measured boot and the launch measurement scale differently

One `snp-aws` image across four vCPU tiers (2, 4, 8 and 32) on observed Milan hosts, and the two quantities behaved in opposite ways:

- the SEV-SNP **launch measurement** differed on *every* tier — four tiers, four distinct digests, as expected from one VMSA per vCPU;

- PCR4 and PCR7 were **byte-identical** on all four.

That is the empirical basis for storing measured-boot registers once per image rather than once per variant: they describe the boot chain, which does not change with the number of virtual processors. It also means a BYOK key policy pinned to PCR4/PCR7 keeps working when the operator resizes the instance, whereas a policy pinned to the launch measurement does not.

A fifth tier, `m6a.4xlarge`, did not come up within the SSM timeout and was recorded in the registry as `capture_skipped_shapes` rather than quietly omitted. An absent tier is refused at deploy time, which is the correct outcome: the alternative is an allowlist that silently does not cover a size the operator can select.

12.7 Supply chain: two pinned values that had been wrong for months

Both were silent, both were the *pin* rather than the verification logic, and both were surfaced only after the bake was made to fail closed (§13).

A commit pin that never existed. The bake pinned `snpguest` at `ec1cc1af26b6...`, labelled `v0.7.0`. A fresh clone of the upstream repository reports that object as absent — not an unreachable reference, an object that is not in the repository at all. So `git checkout` failed, the script’s own supply-chain guard correctly refused to build an unpinned revision, and because the build step was non-fatal the bake completed and shipped an image with no `snpguest` at all. The actual `v0.7.0` tag is `49494ff71b58...`, confirmed by `version = "0.7.0"` in the checked-out manifest.

“ARK” fingerprints that were intermediates. The bundled certificate files are each two certificates, `[intermediate, ARK]`. The pinned fingerprints had been taken from index `[0]` — `CN=SEV-VLEK-Milan` and `CN=SEV-Genoa` — while the verifier compares the *last* certificate of the chain downloaded from AMD, which is the real root. The comparison could therefore never succeed, and because a rejected chain is deleted, **no `snpguest` bake had ever installed an AMD endorsement chain**. That the two pins came from *different* endpoints — one from the VLEK chain, one from the VCEK chain — is the tell that they were copied rather than verified. Client-side verification was never affected: the client also takes the last certificate of the baked bundle, and that has always been the genuine root.

Both are now asserted by test: the pin must equal the last certificate of the bundle, that certificate must be self-signed and named `ARK-*`, and all three SEV-SNP platforms must agree, since a per-generation root is a property of AMD rather than of a cloud.

13 Making the bake fail closed

Both findings above share one cause: a build step that degrades to a warning produces an image that appears complete. Both attestation binaries — `snpguest` for SEV-SNP reports, `nitro-tpm-attest` for NitroTPM documents — are built from source on the bake instance, and each build was individually non-fatal so that its failure text survived. The combination was a fail-open: the bake completed, the image registered, and nothing said it could not attest.

Figure 11 shows the gate that now sits before the bake marker.

14 Measurement capture and the registry

A pin is only evidence if something measured it. Every CVM image is booted after bake and its launch measurement read from the firmware on the instance, never recomputed on the build

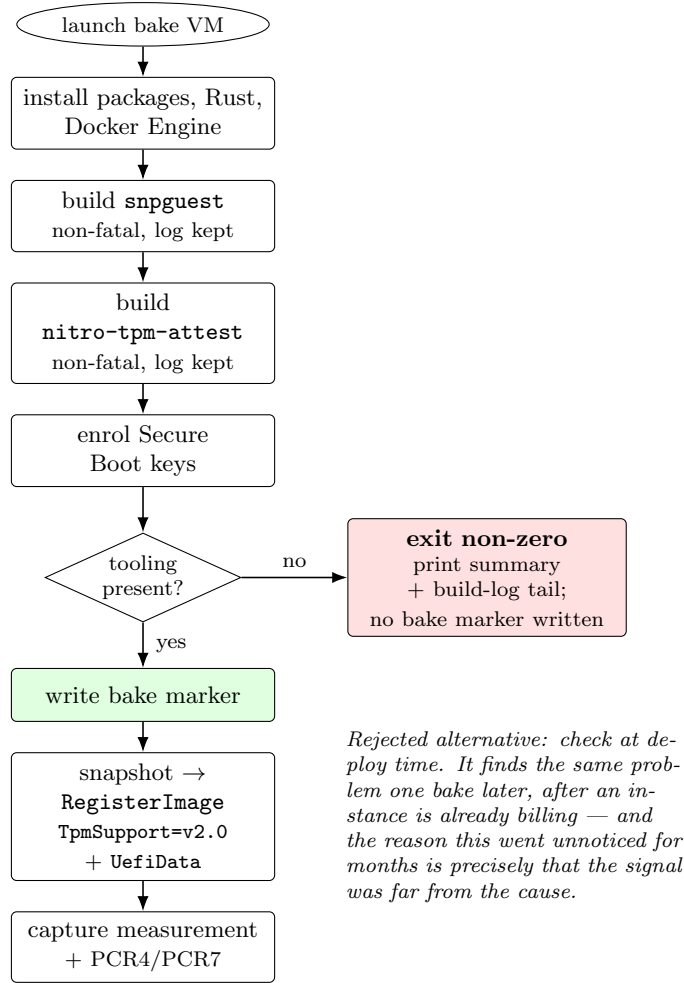


Figure 11: The bake, with the fail-closed tooling gate. Failed builds now leave their logs inside the image rather than in a temporary file the script deleted, which is what made the `snpguest` failure diagnosable at all.

host; hand-entered pins are rejected by test, because a value nobody measured is a claim rather than evidence.

Figure 12 shows the capture walk. The SEV-SNP path is the interesting one: because the launch digest folds in one VMSA per vCPU, a single probe does not generalise, and because the host generation is not selectable the walk has to record what it observed rather than what the instance type implied.

Two platforms additionally record measured-boot registers, and the distinction between them matters. `nitrotpm_pcrs` (`snp-aws`, `gpu-cc-aws`) are SHA-384 and *signed* — they arrive inside a hypervisor-signed document. `vtpm_pcrs` (`gpu-cc-gcp`) are SHA-256 and *unsigned* — the server publishes them about itself, so they are a tripwire and the platform’s real CPU anchor remains its TDX quote. Only these two platforms record PCRs; other CVMs have vTPMs that would answer the probe, but nothing consumes the values and recording them under these names would be a labelling error.

Where a probe costs money, the cost is chosen deliberately. `gpu-cc-aws` has no launch measurement to fold a PCR read into, so it boots its own probe — on a cheap non-GPU shape, because PCR4 hashes the binaries UEFI executed and PCR7 the Secure Boot policy, and neither depends on an attached GPU. The instance type used is recorded in the registry so the assumption is auditable rather than implicit. The bake instance itself cannot be reused for this: it booted the *base* image, so its PCRs measure the boot chain being replaced.

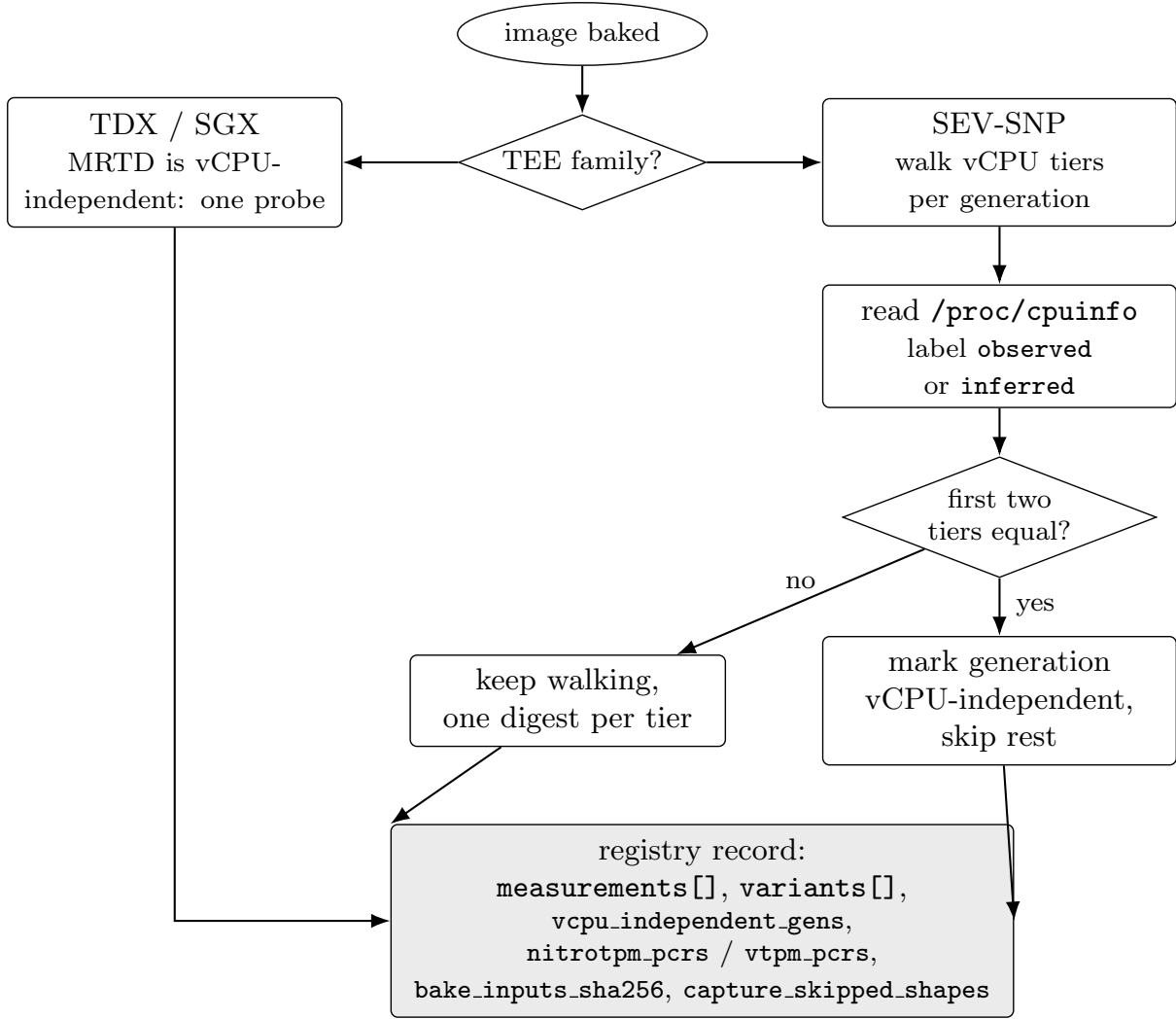


Figure 12: Measurement capture. `vcpu_independent_gens` is only written when every sample in the group was *observed*: two equal digests under two guessed labels are equally consistent with both probes having landed on the same host, which does not establish independence.

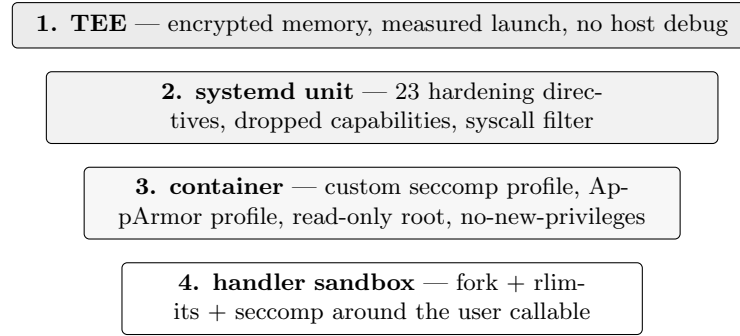
15 Runtime hardening

Attestation says what booted. Hardening is what limits the damage if the workload itself is hostile or compromised — a case attestation says nothing about, since a correctly-measured image can still contain a bad application.

15.1 Three nested boundaries

15.2 The systemd layer

Every platform’s unit applies the same set of directives, and they are asserted by test rather than left to review drift: `NoNewPrivileges`, `PrivateTmp`, `PrivateDevices` (with an explicit `DeviceAllow` for the TEE device only), `ProtectSystem=strict`, `ProtectHome`, `ProtectKernelTunables`, `ProtectKernelModules`, `ProtectKernelLogs`, `ProtectControlGroups`, `ProtectProc` with `ProcSubset`, `RestrictAddressFamilies`, `RestrictNamespaces`, `RestrictSUIDSGID`, `LockPersonality`, `SystemCallArchitectures=native`, `SystemCallFilter`, `CapabilityBoundingSet= (empty)`,



Each layer assumes the one outside it may already be lost. The handler sandbox exists because the innermost thing — arbitrary user code — is the least trustworthy part of the system, and it is the part the operator chose.

Figure 13: Defence in depth inside the TEE. The TEE protects the workload from the host; layers 2–4 protect everything else from the workload.

`AmbientCapabilities= (empty)`, `RemoveIPC`, `KeyringMode=private`, `IPAddressDeny`, `UMask`, and an explicit `ReadWritePaths` allowlist.

`MemoryDenyWriteExecute` is deliberately *not* set: the Python runtime and several native wheels require writable-executable mappings, and a directive that has to be removed the first time it bites is worse than one that was never claimed.

15.3 Container and handler layers

The container runs under a project-specific seccomp profile and an AppArmor profile, both rendered into the image at bake time. AppArmor parse failure is **fatal at bake** — an unparsed profile silently means no confinement, which is the failure mode most likely to go unnoticed, so the bake refuses rather than continuing.

The innermost layer wraps the user’s request handler in a forked child with resource limits and its own seccomp filter, so a handler that leaks memory or attempts an unexpected syscall takes down a request rather than the enclave.

16 The in-TEE audit log

A runtime audit log written by the workload’s own host is worth very little: the party you are defending against can rewrite it. Making it evidence takes three properties, and the third is the one usually missing.

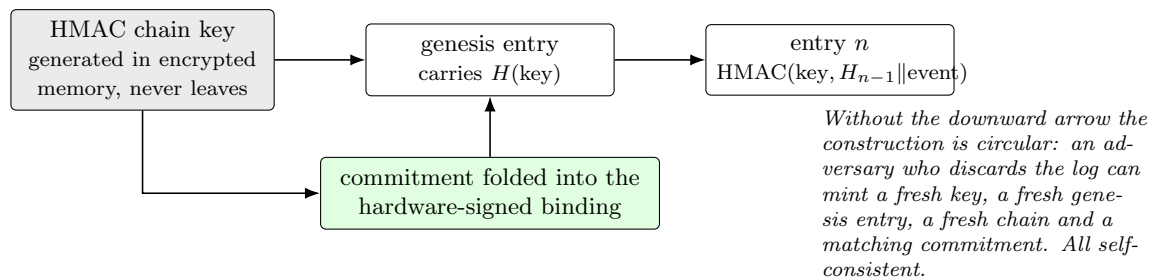


Figure 14: The audit chain. Tamper-evidence comes from the HMAC chain; *anchoring* comes from folding the key commitment into the attestation preimage, so the commitment is under a hardware signature the host cannot forge.

The three properties, and what each buys on its own:

1. **Confidentiality of the key** — it is generated inside encrypted memory and never written out. Buys: the host cannot append convincing entries.
2. **Chaining** — each entry commits to its predecessor. Buys: deletion and reordering are detectable *within* a log.
3. **Anchoring** — the key’s commitment is part of the preimage the hardware signs. Buys: wholesale replacement is detectable, which is the attack the first two do not cover.

Clients fail closed when a server declares no commitment, because an unanchored log is indistinguishable from an honest one. The escape hatch (`TEE_CRAFTER_ALLOW_UNBOUND_AUDIT_CHAIN=1`) exists for development and prints a warning saying exactly what has been given up.

16.1 Export, and what the collector receives

Events are exported on a fixed interval, signed before chaining so a collector cannot be handed a forged sequence, and carry the sequence number, last status, last digest and platform so a gap is visible at the receiving end rather than only in the source. The exporter writes a health file that the request path reads, which is the mechanism behind the fail-closed gate — and the reason the exporter must run inside the TEE, since a health file on the parent instance is a health file the enclave cannot read and an adversary can write.

17 What “unmodified workload” costs

The design choice that the application is not modified is not free, and the honest accounting is short.

A network hop. In persistent mode the attested proxy terminates RA-TLS and forwards over loopback, so every request crosses one extra userspace boundary. For the request sizes this is aimed at — inference, analytics, document processing — that is not the dominant cost, but it is not zero and a latency-critical workload should measure it.

Encryption the workload cannot see. The ECDH plus AES-256-GCM channel is terminated by the platform, so the workload receives plaintext on loopback. That is what makes it unmodified; it also means the workload cannot make per-request authorisation decisions based on the attested peer, because it does not see the attestation. An application that needs that must be modified after all.

Confinement the workload may notice. The layers in Figure 13 deny things applications sometimes do: writing outside declared paths, spawning namespaces, unexpected syscalls. A workload that trips one of these fails rather than degrading, which is correct and is still a migration cost.

Memory encryption overhead. A property of the hardware rather than of this tool, and it varies by platform and access pattern. We have not measured it and do not claim a figure.

No horizontal scaling story. One deployment is one VM or one enclave. Adding a scheduler that can move a workload between hosts would require re-attesting on every move and would change what a pinned measurement means. That is deliberate scope, not an oversight, and it is the limitation most likely to matter for a production adopter.

18 Network and egress

The default posture is deny-all egress, and every hole is enumerated. This is where the cloud differences are largest, because each provider expresses egress in a different object with different failure semantics.

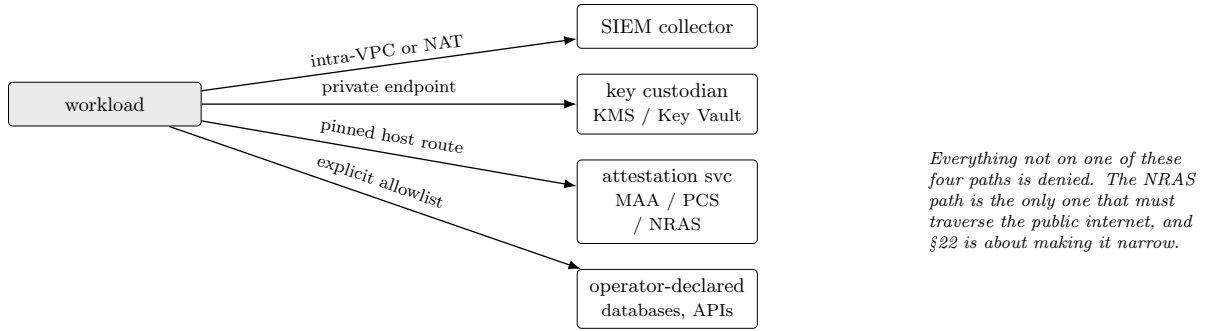


Figure 15: Egress paths. Four categories, each with a different mechanism; the default for anything else is deny.

Two details that are easy to get wrong. First, a bake needs broader egress than a deploy — it installs packages — so the bake’s NAT path is separate and is not left behind for the workload; leaving it in place is possible only via an explicit opt-in that is recorded. Second, SIEM connectivity has four modes (**auto**, **private**, **public**, **none**) and **private** *fails closed* when the chosen collector requires the public internet, rather than silently falling back.

19 Batch mode mechanics

Batch mode has to answer a question persistent mode does not: what did the computation produce, and can a third party believe it?

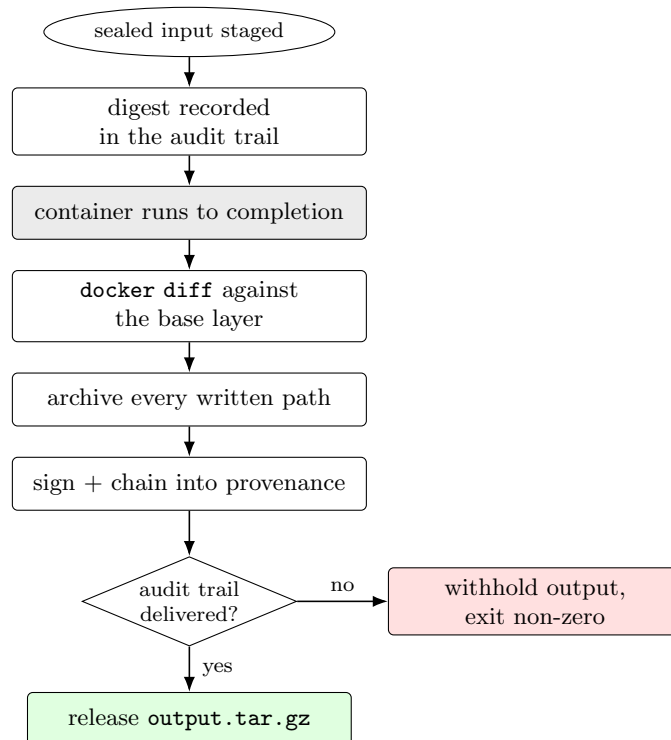


Figure 16: Batch mode. The final gate is the batch analogue of the persistent fail-closed gate: with no delivered audit trail the output is withheld, because an unauditable result is not a result.

Using `docker diff` rather than a declared output directory is a deliberate choice. A declared

directory captures what the author remembered to write there; the filesystem diff captures what the program actually wrote, including the file it wrote to the wrong place. The cost is that transient junk is captured too, which is the right way round for evidence.

On SGX the input and output paths are host-visible and the enclave does not verify them. Integrity for the input rests on the digest in the signed audit trail; the output is written in the clear. Gramine prints a warning on every such run and it is not suppressed.

20 Teardown and cost hygiene

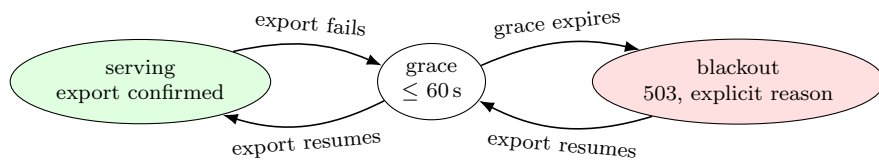
An unattended confidential-computing tool that leaks instances is a liability, so teardown is part of the product rather than an afterthought. Three specifics earned their place by having gone wrong:

- **Deregistering an image does not delete its snapshots.** They keep billing. The bake records the backing snapshot IDs in a ledger at capture time, because after deregistration the association is no longer discoverable from the API — so the only moment the ID is knowable is while the image still exists.
- **Post-destroy secret shred.** Staged secret material is shredded after destroy by default.
- **Verify the spend actually stopped.** A destroy that reports success is not evidence that nothing is running; the check is a fresh **describe** call afterwards, not the exit code of the thing that was supposed to do the deleting.

That last point generalises beyond cost. A teardown script whose flags failed to parse exits non-zero having deleted nothing, and an operator reading a wall of output can easily miss it.

21 Continuous attestation, and failing closed

On persistent deployments a sidecar re-attests on a fixed interval and exports signed, hash-chained events to a SIEM. The in-TEE request path refuses traffic while export cannot be confirmed — returning an explicit blackout error rather than serving unaudited requests. Figure 17 is the state machine.



The gate is only meaningful because the exporter runs inside the TEE. On AWS Nitro a host-side sidecar would write its health file on the parent instance, which the enclave cannot read — so the gate would pass every request while proving nothing. Verified by pointing the collector at an unresolvable host: the in-enclave exporter reported failure, the gate armed, and the deploy exited non-zero rather than reporting success.

Figure 17: The SIEM fail-closed gate. Default is fail-closed; `TEE_CRAFTER_SIEM_FAIL_OPEN=1` reverts it and is recorded in the ledger when set.

22 Egress posture for GPU attestation

NVIDIA’s Remote Attestation Service is reachable only over the public internet; there is no cloud service endpoint. The production default is a strict CIDR allowlist, and that used to

be unsatisfiable: NVIDIA publishes no ranges, so a strict deploy created no rule and GPU attestation failed. The only working configurations were a broad-internet dev hatch or hand-pinned ranges.

The stated reason — that the endpoint sits behind a CDN whose addresses rotate — was wrong. Measured: `nras.attestation.nvidia.com` answers with a *single* A record, 34.120.45.54, identical from two independent resolvers, inside 34.64.0.0/10, which is the range Google Cloud assigns to global external Application Load Balancers. That is an anycast VIP belonging to one load balancer — the opposite of a rotating CDN edge.

Strict mode therefore resolves the hostname at deploy time and pins the resulting host route: narrow, auditable, recorded in the audit trail, and self-healing on the next deploy. Exercised on hardware: a `gpu-cc-azure` deploy in `eastus2` resolved, pinned, and passed GPU attestation with no broad-internet rule.

Two caveats remain. The pin is point-in-time: NVIDIA gives no guarantee about that address, so a long-lived deployment can begin failing attestation with a connection timeout if the VIP moves, and the fix is a redeploy. And moving verification to the SDK's *local* verifier is not an escape — it swaps one hostname for two (`rim.attestation.nvidia.com` and `ocsp.ndis.nvidia.com`) and its OCSP check has no skip flag, so fully offline GPU attestation would need a patched verifier.

23 Evidence produced

Every deployment emits an evidence bundle rather than a log. The pieces:

- **Audit-evidence ledger** — one verdict per check (`pass` / `fail` / `warn` / `not_applicable`), hash-chained and signed, renderable as JSON, text, Markdown or HTML, and gateable in CI on a required-check list.
- **Build provenance** — hash-chained, Ed25519-signed, with a SLSA in-toto statement and DSSE envelope alongside.
- **The verifier's own reasoning** — the client writes its verdict to stdout and its reasoning to stderr, and both are retained. Keeping only the verdict meant an auditor could see that attestation passed but not what was checked; worse, on a failure the one artefact that explains why was the artefact being discarded.
- **Batch output** — a signed archive of every file the container wrote, obtained by differencing the container filesystem.

24 The audit-evidence ledger

Every deployment emits a ledger of verdicts rather than a log of activity. The distinction matters to a reviewer: a log says what happened, a ledger says what was *checked* and what the answer was, including checks that could not be performed.

The catalogue holds **130 checks in 16 families**. Family sizes are themselves informative — they show where the risk was judged to be.

Four verdicts, and the fourth is the one that makes the ledger honest:

- **pass** — observed value matched the expected value.
- **fail** — it did not. CI must fail closed.
- **warn** — evidence could not be collected: a missing IAM permission, an unreachable probe. *Not* a pass.

Family	Count	What it covers
DH	19	Dev hatches. Every switch that weakens a default is a check, so using one is visible rather than merely possible.
BYOK	14	Key release: provider resolved, unwrap mode, allowlist size, key-id tail, tmpfs relocation, KMS policy diff.
PDR	11	Platform data residency and jurisdiction.
ATT	10	Attestation: issuer allowlist, TLS binding, quote validity, report kind, signature algorithm.
IAC	9	Terraform static checks and apply outcomes.
PC	9	Preconditions: platform recognised, flow detected, build dir writable, CLI version recorded.
PKG	8	Packaging hygiene and measurement computation.
CT	7	Container hardening.
PROV	7	Provenance chain and signature.
SIEM	7	Export wiring and the fail-closed gate.
EGR	6	Egress allowlists actually applied.
TEAR	6	Teardown completed and verified.
VLN	6	Vulnerability thresholds.
DEP	5	Dependency pinning.
IAM	5	Least privilege on the deployer and instance roles.
RES	1	Region within residency policy.

Table 8: The 130-check catalogue. That DH is the largest family is deliberate: the most likely way a deployment is weaker than it looks is that somebody set an environment variable.

- `not_applicable` — the check does not apply to this platform or configuration.

Collapsing `warn` into `pass` is the single easiest way to make a compliance report worthless, because the checks most likely to be uncollectable are the ones guarding the least-privileged paths.

The ledger is hash-chained and signed, renders to JSON, text, Markdown and HTML, and `verify-provenance --ledger` can gate CI on a required-check list. It also carries the verifier client’s own stderr — its reasoning, not just its verdict — because retaining only the verdict means an auditor can see that attestation passed without seeing what was checked, and on a failure the one artefact that explains why is the one being discarded.

25 Compliance mapping

The compliance engine maps observed evidence to control requirements across **14 frameworks and 110 controls**, driven by **39 evidence keys** extracted from the run rather than from a questionnaire.

25.1 The responsibility model is the load-bearing part

Every control carries a responsibility label, and this is what stops the report from overclaiming. Taking HIPAA’s technical safeguards as the worked example: of seven mapped controls, five are **PRODUCT** and two are **SHARED**. None are claimed as wholly satisfied by the tool.

Each control names its supporting `evidence_keys`, drawn from the 39 observed items — among them `attestation_tls_binding`, `container_digest_pinning`, `continuous_attestation`, `audit_log_tamper_evidence`, `measured_boot_vtpm`, `dual_attestation_cpu_gpu`, `dependency_hash_pinning`, `deployer_least_privilege` and

Framework	Edition mapped	Controls
NIST 800-53	Rev 5 (Sep 2020)	15
SOC 2 Type II	2017 TSC, 2022 points of focus	10
ISO/IEC 27001	2022 Annex A	9
PCI DSS	v4.0	9
GDPR	Regulation (EU) 2016/679	8
GLBA Safeguards Rule	16 CFR Part 314 (2023)	8
HITRUST CSF	v11	8
NIST CSF	2.0 (2024)	8
HIPAA Technical Safeguards	45 CFR 164.312	7
CSA Cloud Controls Matrix	v4.0	7
EU AI Act	Reg. 2024/1689 (selected)	6
EU NIS2 Directive	2022/2555 (selected)	6
ISO/IEC 27701	2019 privacy extension	5
CCPA / CPRA	Cal. Civ. Code 1798.100 et seq.	4
Total		110

Table 9: Frameworks mapped. Breadth here is cheap and depth is not, so the claim is deliberately narrow: these are the controls a confidential-computing deployment can produce *technical* evidence for, not a full certification package.

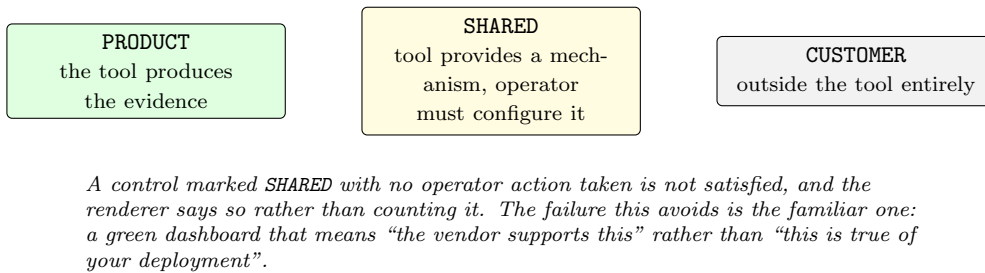


Figure 18: The three-way responsibility split. Controls carry **evidence_keys** naming which observed evidence items support them, so a claim can be traced to a measurement.

build_reproducibility. Where an evidence key is absent from a run, the controls depending on it degrade rather than passing on the strength of the others.

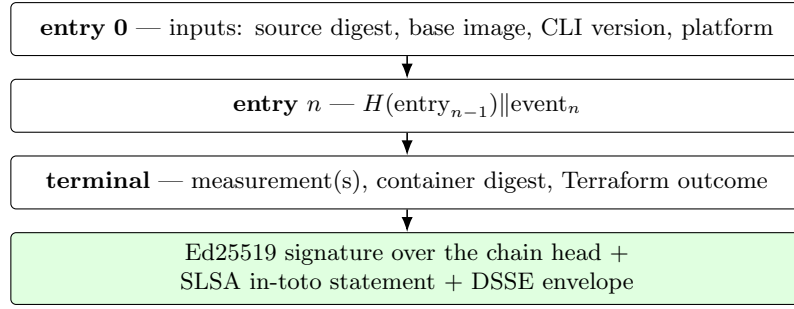
25.2 Data residency

Region selection is checked against a jurisdiction policy before anything is provisioned (RES-001, and the PDR family). This is a pre-deploy gate rather than a post-hoc report, for the obvious reason: a residency violation discovered after apply has already happened.

26 Build provenance and supply chain

The provenance trail answers “what went into this deployment” with a chain rather than a claim.

The bake side has its own version of this: the rendered setup script’s digest is recorded alongside the captured measurement, so a deployment can detect that its image predates a change to something baked into it.



Signing keys come from `TEE.CRAFTER.PROVENANCE.SIGNING_KEY` or a file; an ephemeral key is available for local work but must be opted into, so an unsigned-by-accident build does not silently look signed.

Figure 19: Provenance chain. **verify-provenance** recomputes every link and checks the signature; the build directory’s own integrity is verified before a **deploy-from-build** resumes it.

27 Threat model

Stating what is and is not defended against, because a TEE deployment invites assumptions in both directions.

28 Testing strategy, and the ways tests lie

The suite is 4,944 tests. Test count is a poor proxy for assurance; the following failure modes were each observed in this suite and each produces a passing test that verifies nothing.

1. **Passing on the wrong rejection.** A test asserts that a hostile input is refused. It is — by a different check than the one under test. Removing the intended check leaves the test green. The fix is to assert on the specific reason, not merely on refusal.
2. **Asserting on a constant nothing reads.** A build-time pin was emitted into a client and consumed by no code. The test verified it was emitted. Both the constant and the test were describing an enforcement that did not exist.
3. **Satisfied by a comment.** Source-order assertions over templates that document their own ordering in prose: a search for `verify-gpu-nras-token(` matches the sentence describing the call. Mitigation: strip comment lines before asserting.

A fourth mode is structural rather than a test defect: **a verifier check can be correct, fully covered, and inert** because the producer never emits the input it checks. Instance: the `gpu-cc-gcp` client compares a vTPM PCR bundle its image cannot produce, because the bake omits the TPM tooling the runtime invokes. Unit coverage of both sides is complete and the composition is non-functional.

Mitigation is cross-file invariants rather than additional unit tests:

- a runtime template that invokes a binary implies its bake installs it;
- a pin claiming to be a root certificate must equal the last certificate of its bundle and be self-signed;
- a documented environment variable must have a read site in the code.

Adversary	Defended?	By what, or why not
Cloud operator reading guest memory	Yes*	Memory encryption on the eight CVM/enclave platforms. * <i>Not</i> on <code>gpu-cc-aws</code> , which has no CPU-TEE.
Malicious host substituting a different image	Yes	Launch measurement pinned from a booted capture; mismatch is fatal.
Malicious host replaying a captured report	Yes	Channel binding: the report commits to the session’s ephemeral key. On Azure this runs through the paravisor’s AK (§6.2).
Host discarding the audit log	Yes	The chain-key commitment is folded into the hardware-signed binding, so a replacement log cannot be self-consistent.
Host silencing the SIEM	Yes	Fail-closed gate; the exporter runs inside the TEE so its health cannot be forged from outside.
Instance credentials stolen from IMDS	Partly	Key release upgraded to measured-boot-gated where the custodian supports it; identity-gated otherwise, and labelled as such.
Compromised workload exfiltrating data	Partly	Deny-all egress with an enumerated allowlist. A workload permitted to reach a database can send data to that database.
Malicious workload image	No	Attestation proves <i>which</i> image ran, not that it is benign. The digest binding makes it attributable.
Compromised upstream dependency	Partly	Pinned versions and commit pins, vulnerability gating — but §12.7 shows a pin can be wrong and quiet.
Silicon vendor or attestation service	No	They are the root of trust. On <code>tdx-azure</code> that root is Microsoft, not Intel.
Physical attacker with the machine	No	Out of scope for all of these platforms.

Table 10: Threat model. The rows marked “Partly” are where honest description matters most, because each is often presented as fully solved.

Each invariant was validated by reintroducing the defect and confirming failure.

Beyond the suite: offline verification against vendor certificate authorities, a live fetch against Intel’s Provisioning Certification Service, `terraform validate` across all ten templates, and hardware runs.

29 A worked evidence bundle

What a reviewer actually receives, for one `snp-aws` batch deploy. The point of showing it is that every line is traceable to something measured rather than asserted.

Verdicts from a representative run:

- **VLN** — no fixable CRITICAL/HIGH; 17 unfixed upstream (3 critical, 14 high) recorded with the note that no patch exists. The distinction is the verdict, not a footnote.
- **PKG** — container digest computed and bound into the attestation preimage.
- **IAC** — ten-template static checks; egress materialised as deny-all with VPC-local 443 only.
- **ATT** — launch measurement matched one of four captured vCPU tiers; the tier that was not captured is named rather than omitted.

```

builds/<id>/
audit/      ledger.json .txt .md .html + .sig
provenance/ build_provenance.json + .sig + pubkey
slsa/       in-toto statement + DSSE envelope
compliance/ 14 frameworks rendered
byok/       byok.json (redacted) + byok.env.public
siem/       config + egress allowlist
probes/     in-guest probe stdout/stderr
client_output.json the verdict
client_stderr.log  the reasoning
output.tar.gz  signed, batch only

```

Figure 20: The bundle. `byok.env` — the only file with secret material — is relocated to `tmpfs` and the disk copy shredded, so it is absent here by design.

- **BYOK** — provider resolved, `unwrap` recorded as `aws_nitrotpm_recipient`, and — after the fix in §10.2 — the key-policy pin recorded as a verdict rather than skipped silently.
- **TEAR** — 29 resources destroyed, confirmed by a fresh `describe` showing zero instances.

The registry record behind that run is worth quoting because it shows the two quantities behaving differently in the same file: four distinct launch measurements across `m6a.large` / `xlarge` / `2xlarge` / `8xlarge`, one identical `nitrotpm_pcrs` pair for all four, every variant carrying `cpu_gen: milan` with `cpu_gen_source: observed` and `cpu_model: AMD EPYC 7R13`, and `capture_skipped_shapes: ["m6a.4xlarge"]` — a tier whose SSM agent did not come up in time, recorded rather than quietly dropped.

That last field is the one a reviewer should look for. An allowlist that silently omits a shape the operator can select is worse than one that says so, because the failure appears later as an unexplained measurement mismatch.

30 Verification status

Eight of the ten backends have been deployed to real hardware and verified end to end. The two that have not — `gpu-cc-aws` and `gpu-cc-gcp` — are blocked on accelerator supply rather than on code: `p5.4xlarge` returned `InsufficientInstanceCapacity` in all seven availability zones of two regions, on-demand and spot, and the Google Cloud project used for validation has no H100 quota metric at all, which is a stronger statement than a limit of zero — the shape cannot be requested, so there is no limit to file an increase against.

Beyond hardware: 4,944 automated tests, offline verification against real vendor certificate authorities, a live fetch against Intel’s production Provisioning Certification Service, and `terraform validate` across all ten platform templates.

That test count deserves a caveat, because a number like it invites more confidence than it earns. Several tests in this suite were found to pass for the wrong reason — asserting on a rejection raised by a different check than the one under test, or on a constant no code reads. A further mode: **a test can be satisfied by a comment**. These templates explain their own ordering in prose, so an assertion looking for `verify_gpu_nras_token(` matched the sentence describing the call rather than the call. The fix — strip comment lines before asserting — is small; noticing was not, and it happened three separate times.

A related hazard: a verifier-side check can be correct, fully covered, and inert because the server never presents the input it checks for. That was the situation on `gpu-cc-gcp`, where the client compares a vTPM PCR bundle the image could not produce, because the bake never installed `tpm2-tools` even though the runtime shells out to it. Nothing failed until hardware would have.

31 Limitations

We state these rather than leave them to be discovered.

- `tdx-azure` roots in Microsoft, not Intel. This is a property of the platform, not a configuration choice.
- `gpu-cc-aws` has no CPU-TEE. Its boot chain is attested and its host memory is not encrypted; measured boot is a real but strictly weaker property than a confidential VM, and the deploy requires an explicit acknowledgement before it will proceed.
- Key release on the AWS CVM platforms binds measured boot, not the SEV-SNP launch measurement. AWS publishes no condition key for the latter.
- The NRAS address pin is point-in-time, and fully offline GPU attestation is not possible without patching NVIDIA's verifier.
- Batch input and output on SGX pass through host-visible paths that the enclave does not verify. Integrity for the input rests on the digest recorded in the signed audit trail; the output is written in the clear. Gramine prints an explicit warning on every such run, which is not suppressed.
- The SIEM fail-closed gate has been exercised against a dark collector on two backends, not all ten.
- On Google Cloud SEV-SNP, a correct pin can go stale without the image changing (§12.7 is the supply-chain analogue; the measurement case is above).

32 Reproducibility

Machine images are baked from a rendered setup script whose digest is recorded alongside the captured measurement, so a deployment can detect that its image predates a change to something baked into it — a failure mode that is otherwise invisible, since a stale image looks exactly like a fresh regression.

The same hazard applies to the tool itself. The CLI runs from a container built from the source tree rather than bind-mounting it, and an image-reuse path that returns early whenever the tag exists means editing code and re-running silently exercises the *previous* build. Observed effect: two runs completing cleanly while executing deleted code, with an error message attributable to an unrelated cause. The fix is a source fingerprint compared against a label on the image, covering every file that gets baked in, not only the Python.

Build provenance is hash-chained and signed, and a single command verifies the chain. The measurement registry ships one record per image, each captured from a booted instance.

33 The deploy, phase by phase

Each phase has a gate. This section states what each refuses.

33.1 Phase 1 — build

Builds the workload image with BuildKit and records its digest. The vulnerability gate runs here: production default blocks *fixable* CRITICAL and HIGH findings, on the reasoning that an unfixable finding is information rather than a decision — there is no patch to apply, so blocking only prevents deployment without improving security. VLN-003 and VLN-004 carry the thresholds; `TEE_CRAFTER_VULN_STRICT=1` restores zero tolerance. The failure it catches is a persistent-shaped image deployed with `--batch`, which runs until the timeout.

33.2 Phase 2 — package and measure

Computes what is computable at build time (Nitro PCR0, SGX MRENCLAVE), renders the verifier client with the pins the registry holds, and stages the runtime modules. The gate: a sealed-secrets or BYOK deploy against an image with *no* registry record is refused. An unpinned TEE is one whose launch state nothing has vouched for, and releasing secrets to it is the failure this whole mechanism exists to prevent.

This is also where the client is rendered rather than hand-written, which matters for a subtle reason: the verifier’s expectations come from measured values, so there is no path by which an operator’s typo becomes an accepted pin.

33.3 Phase 3 — generate Terraform

Renders one of ten platform templates. Static checks run before apply (IAC family): ingress rules are inspected for over-broad sources, Secure Boot and vTPM posture is asserted where the platform supports it, and the workload egress allowlist is materialised into the cloud’s own object.

33.4 Phase 4 — apply

Terraform apply, with a bounded retry for the genuinely transient failures and partial-state cleanup for the rest. Two gates fire immediately after:

- **Residency** (RES-001, PDR-*) — actually checked *before* apply, because a jurisdiction violation discovered afterwards has already happened.
- **BYOK key pinning** — the key policy is narrowed to the role Terraform has just created, which is the first moment that role’s exact ARN exists. Fail-closed. §10.2 is about the flows where this was missing.

33.5 Phase 5 — verify and collect

The operator’s client connects and verifies independently. Probes collect in-guest evidence over the platform’s remote-exec channel. Cloud audit logs are queried for the privileged actions the deploy itself took. The ledger is rendered and signed; on batch, the output archive is released only if the audit trail was delivered.

34 Continuous attestation internals

Attestation at connect time proves the state at connect time. For a long-lived service that is not enough, so the persistent flow re-attests.

Each tick re-reads fresh hardware evidence rather than caching: on **snp-aws** a new SNP report from `/dev/sev-guest`, on **snp-azure** a fresh vTPM read. A cached report would make the whole exercise theatre, since the question being asked is whether the machine is *still* what it claimed to be.

Events are hash-chained and exported to one of four SIEM backends (**syslog-cef**, **splunk-hec**, **datadog**, or **none**), signed before chaining so a collector cannot be fed a forged sequence.

35 Operational runbook: reading a failure

Common failures and their actual causes.

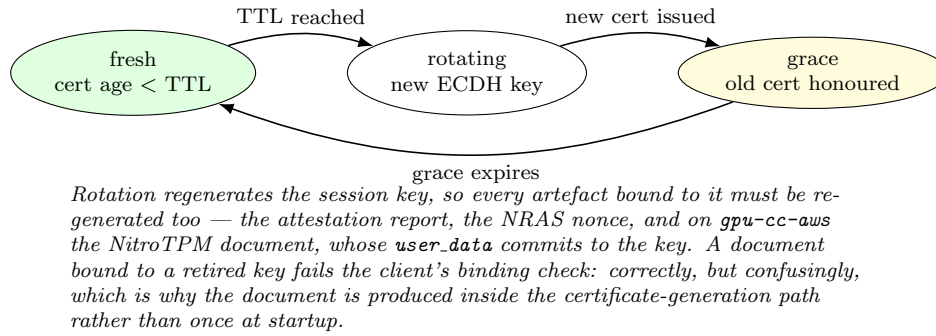


Figure 21: Certificate and key rotation. `TEE_CRAFTER_CERT_TTL_SEC` and `TEE_CRAFTER_REATTEST_INTERVAL_SEC` control the cadence; the grace window exists so an in-flight client is not cut off mid-request.

36 Cost model for hardware verification

Hardware verification is metered, which constrains method. A bake is approximately one instance-hour of a small shape plus one probe instance per vCPU tier; a deploy ranges from minutes to hours by run mode.

- **Batch queries per session.** Enumerate installed binaries, PCR values, certificate chains and service states in one remote-exec session rather than one per question.
- **Probe on the cheapest shape that answers the question.** PCR4/PCR7 are properties of the boot chain, so they are read on a general shape rather than a GPU shape — two orders of magnitude cheaper for an identical answer. The shape used is recorded so the assumption is auditable.
- **Confirm termination by query, not exit code.** A destroy that reports success is not evidence that nothing is running; a fresh `describe` is.
- **Match run mode to workload shape.** A server-shaped image deployed with `--batch` produces a container that never exits and bills until the timeout. The build phase warns on this heuristically.

37 Positioning

Three families of approach exist, and the trade-off is consistent.

Modify the application. Link an attestation SDK, speak the protocol, handle key release in-process. Maximum control, and it puts confidential computing behind an application rewrite — which in practice means it does not happen for existing workloads.

Deploy into a confidential VM and stop. Every cloud offers this and it is genuinely useful: memory is encrypted. But nothing verifies the launch state, so the guarantee rests on the provider's assurance rather than on evidence, and there is no artefact a reviewer can check.

Managed confidential-container services. The provider handles attestation, and the attestation verdict comes from the provider. That is a reasonable trade for many teams; it is also the shape that makes `tdx-azure`'s root of trust Microsoft rather than Intel, and it is worth knowing which one you have.

This work sits in the second-and-a-half position: unmodified workloads, but with independent verification and a retained evidence bundle. The cost is honesty about where that independence stops — two of ten backends root in a service rather than in silicon, and one has no CPU-TEE at all.

Symptom	What it usually is
Measurement mismatch on <code>snp-gcp</code>	Host firmware moved under you. Re-measure the same image before suspecting the image. Still a mismatch — never widen the allowlist.
Measurement mismatch after an instance resize	Expected on SEV-SNP: the digest folds in one VMSA per vCPU. The registry should already hold that tier; if it does not, the tier was skipped at capture.
<code>AccessDenied</code> from KMS with a document attached	Check the document’s age first — five minutes is the limit — then the PCR bank. A SHA-256 value in a policy denies the legitimate caller.
KMS <i>succeeds</i> when you expected refusal	The policy has a conditional <code>Allow</code> and no <code>Deny</code> . See §10.1.
Attestation passes but the deploy exits non-zero	Usually the fail-closed SIEM gate. Check whether the collector is reachable <i>from inside</i> the TEE, not from your workstation.
NRAS attestation times out	Re-resolve the hostname before looking for a code regression; the pinned address may have moved. A redeploy re-pins.
Empty vTPM PCR bundle on <code>gpu-cc-gcp</code>	The image predates <code>tpm2-tools</code> being installed by that bake. Re-bake.
<code>SecureBoot disabled</code> on a fresh AMI	<code>RegisterImage</code> dropped <code>UefiData</code> . It must be passed explicitly.
Bake completes, image cannot attest	Only possible on images predating the tooling gate (§13). Check <code>/var/log/tee-crafter/*-build-failed.log</code> inside the image.
Edits appear to have no effect	The CLI image is stale. The fingerprint check should catch it; if it was skipped, that is why.

Table 11: Failure triage. In most rows the tool’s verdict is correct and the first interpretation is not, which is the case for retaining the verifier’s reasoning alongside its verdict.

38 Future work

Ordered by how much they would change the guarantee rather than by effort.

- **A launch-measurement condition key for SEV-SNP on AWS.** Today the strongest available binding on those platforms is measured boot. This is a provider gap, not an implementation one.
- **Offline GPU attestation.** NVIDIA’s local verifier still requires OCSP with no skip flag, so an air-gapped GPU deployment needs a patched verifier. Baking the RIMs removes one of the two hostnames, not both.
- **The Milan digest for `snp-azure`.** Needs luck rather than engineering: Azure schedules the confidential v5 families on either silicon and offers no equivalent of `min_cpu_platform`.
- **Exercising the fail-closed SIEM gate on all ten backends.** The procedure needs no extra spend — both observations come from a run that is already up — so this is discipline rather than budget.
- **Path coverage as a first-class test dimension.** §10.2 was a control present on one flow of three. The generalisable countermeasure is to assert that every deploy flow reaches each fail-closed gate, which no unit test does today.

A Platform capability matrix

Platform	Mem. enc.	Launch meas.	Measured boot	Attested key rel.	Hardware verified
nitro-aws	yes	PCR0 (build-time)	—	yes (PCR0/1/2)	yes
snp-aws	yes	yes (per vCPU)	PCR4/7	yes (PCR4/7)	yes
snp-azure	yes	yes (per vCPU)	vTPM	yes (SKR)	yes
snp-gcp	yes	yes (per vCPU)	—	configurable	yes
sgx-azure	yes	MRENCLAVE	—	no	yes
tdx-azure	yes	via MAA	vTPM	yes (SKR)	yes
tdx-gcp	yes	MRTD	—	configurable	yes
gpu-cc-azure	yes	yes	vTPM	yes (SKR)	yes
gpu-cc-gcp	yes	MRTD	vTPM (unsigned)	configurable	no
gpu-cc-aws	no	—	PCR4/7	yes (PCR4/7)	no

Table 12: Capability matrix. “Configurable” means the provider supports an attestation condition that the operator must add; the shipped default is identity-gated and labelled as such.

B Configuration surface

126 environment variables have a real read site. They fall into five classes, and only the first two should ever appear in a production configuration:

1. **Deployment parameters** — platform, region, instance shape, image id, BYOK and SIEM selection. Each also has a CLI flag.
2. **Hardening knobs** — opt-in strictness: `SNP_REQUIRE_SMT_DISABLED`, the `SNP_EXPECTED_*` pins, `VULN_STRICT`, the `EXPECTED_*_PCRS` overrides.
3. **Escape hatches** — 21 switches that weaken a default, each recorded in the ledger when set. Enumerated in `docs/security.md` §19.
4. **Wrapper-owned** — set by the CLI for its own container and not inherited from the host.
5. **Transcript markers** — *not* inputs. `TEE_CRAFTER_MEASUREMENT=<hex>` and `TEE_CRAFTER_PCR4=<hex>` are lines a capture snippet *prints* for the bake to parse. Mistaking one for a setting is an easy error, which is why the generated index in `docs/cli_reference.md` lists read sites rather than names alone.

C Glossary

ARK / ASK / ASVK AMD Root Key, and the signing keys beneath it for VCEK and VLEK chains respectively.

VCEK / VLEK AMD’s per-chip / per-tenant endorsement keys that sign an SEV-SNP attestation report.

MEASUREMENT The SEV-SNP launch digest: SHA-384 over initial guest memory, including one VMSA per vCPU.

MRTD / MRENCLAVE The Intel TDX and SGX equivalents.

PCR A TPM Platform Configuration Register: a hash chain of what the boot chain executed. *Not* a launch measurement.

COSE_Sign1 The CBOR-encoded single-signer structure AWS uses for both Nitro Enclaves and NitroTPM attestation documents.

EAT Entity Attestation Token; NVIDIA’s NRAS returns one as a JWT.

MAA Microsoft Azure Attestation, whose signed verdict is the attestation on `tdx-azure`.

SKR Secure Key Release: Azure Key Vault releasing a key against an attestation-bound policy.

DEK Data-encryption key — the thing BYOK protects.

RA-TLS TLS whose server certificate carries attestation evidence in X.509 extensions, so no application protocol change is needed.

Paravisor The Hyper-V component that mediates the guest’s access to SEV-SNP/TDX facilities on Azure, and which fixes `REPORT_DATA`.

Acknowledgement of scope

This report describes a system built and validated by one person against real cloud hardware, over a bounded budget. Where a property has not been observed, it is listed as unobserved rather than inferred. The companion document `docs/pending.md` is the exhaustive version of that list, and `docs/security.md` §19 enumerates every configuration switch that weakens a production default.